autentia

DEVOPS

GUÍA COMPLETA



Guía para directivos y técnicos

V.6

FORMA PARTE DE UN EQUIPO AUTÉNTICO



Si buscas empleo y crees que encajarías con nuestra filosofía, puedes ver toda la información detallada en nuestra web.

www.autentia.com/ofertas-de-empleo/

NOS PUEDES ENCONTRAR EN:

YouTube

Encuentra en el canal de **Autentia en Youtube** todos los vídeos de las conferencias tecnológicas más relevantes del sector del desarrollo.

Adictos al trabajo

¡Compartimos conocimiento! Encuentra cientos de tutoriales gratuitos relacionados con el desarrollo y las metodologías ágiles, entre otros.

autentia LIBROS &GUÍAS

Descarga las numerosas guías técnicas desarrolladas por el equipo de Autentia sobre front, back, devops, etc. Y descubre los libros escritos por nuestro CEO, Roberto Canales.

TAMBIÉN ESTAMOS EN:

Autentia

Autentia Real Business Solution, S.L.

@autentia

@autentia_team

DEVOPS

Guía completa

Este documento forma parte de las guías de onboarding de Autentia. Si te apasiona el desarrollo de software de calidad ayúdanos a difundirlas y anímate a unirte al equipo. Este es un documento vivo y puedes encontrar la última versión, así como el resto de partes que completan este documento, en nuestra web.
www.autentia.com/libros/



Esta obra está licenciada bajo la licencia Creative Commons Attribution ShareAlike 4.0 International (CC BY-SA 4.0)

El interés por DevOps (la combinación de **“desarrollo” y “operaciones”**) ha continuado creciendo de manera exponencial hasta convertirse en uno de los anhelos más profundos del negocio digital.

Pero ¿cuál es la motivación real que nos impulsa a incorporar este tipo de prácticas?, ¿cuáles son los detalles que tenemos que tener en cuenta para incorporarlas? y ¿cuáles son las mayores barreras que nos podemos encontrar en el camino?

Estas preguntas e inquietudes nos han impulsado a formular esta guía que nos permitirá, con un enfoque cercano y práctico, aterrizar todo lo que CEOs, CIOs, arquitectos y desarrolladores necesitan saber para **entender la importancia de la cultura DevOps y cómo con ella se puede contribuir** a la entrega de valor al cliente y negocio.

En un mundo cada vez más acelerado y centrado en el cliente, se hace necesario hacer **entregas continuas de software, testar la respuesta del usuario y tener capacidad de entender y adaptarse a sus necesidades.**



Trabajar bajo una filosofía DevOps facilita la **recuperación** en caso de escenarios de contingencia, al tener perfectamente identificados los activos de software y hardware y automatizadas las tareas rutinarias.

En el mercado podemos ver ofertas de perfiles DevOps polarizados en conocimientos de automatización de entornos y configuración en la nube. En la primera parte de esta guía veremos una perspectiva integral.

“Una de las claves para garantizar la calidad en las entregas es reducir la intervención humana a la hora de instalar entornos, empaquetar, testar, entregar o incluso volver al estado anterior en caso de problemas.”

En la segunda parte de esta guía, el objetivo es tratar de **describir las piezas fundamentales** que puede tener una infraestructura moderna, entender la complejidad que estas pueden añadir y cómo prácticas como infraestructura como código son imprescindibles, se pueden gestionar y escalar sistemas muy estables. Partiendo de este punto, tener una infraestructura inestable no solo daña intangibles, como la imagen de marca, sino que provoca pérdidas directas.

¿Cuánto cuestan nuestros proveedores?, ¿cuánto cuesta el hardware?, ¿y las licencias de software? Si no sabemos responder, es probable que estemos pagando plata a precio de oro. Los proveedores de infraestructura en la nube cuentan con **calculadoras** que nos van a servir para estimar cuánto nos va a costar todo. Son complicadas de utilizar y requieren conocer si todo o parte de nuestro sistema puede adaptarse a este nuevo paradigma.

No conocer esta realidad, no nos exime de vivir en ella, y aprovecharnos de sus ventajas puede suponer no sólo un **ahorro en costes**, sino una oportunidad para mejorar la imagen corporativa, la atracción de talento e incluso una ventaja competitiva frente a nuestra competencia.



En la tercera parte de esta guía, se profundiza en los conocimientos que cualquier desarrollador debería tener acerca de la **administración de sistemas**. Sin estos conocimientos, nuestro sendero hacia las grandes cimas de DevOps terminaría aquí. Aunque, normalmente, hay personal especializado a cargo de las infraestructuras, es necesario conocer lo indispensable para mantener nuestro **entorno de trabajo preparado** para la acción y estar al tanto de las características que los sistemas operativos nos ofrecen.

Uno de los temas más importantes a este respecto es la **virtualización de entornos**. Gracias a ella, podemos reproducir con fidelidad las condiciones para el desarrollo o el despliegue de una aplicación de forma sencilla. En los últimos años han surgido diferentes enfoques y tecnologías destinados a facilitarnos las cosas que es imprescindible conocer.

A partir de ese momento, una vez preparado el terreno, la guía irá profundizando en diferentes herramientas y tecnologías relacionadas con la filosofía DevOps, como contenedores, herramientas de provisión, configuración y despliegue, IaC, orquestadores de contenedores y más.

DEVOPS

Guía completa

Índice

Parte 1: Cultura de entrega de valor

- Ideando el producto
- El flujo de desarrollo software
- Empezando con el desarrollo
    - El uso del IDE
    - Git
    - Entorno local
- En local funciona, y ahora qué...
- Entorno de integración
- Integración Continua (CI)
- Despliegue continuo (CD)
    - El planteamiento de GitOps
- Beneficios de la integración y la entrega continua (CI/CD)
- Infraestructura como código (IaC)
- Configuración como código (CaC)
- Beneficios de la IaC y la CaC
- Entornos de preproducción
- Entorno de producción
    - Infraestructura y servicios
    - Características de un entorno de producción
- DevOps, DevOpsSec y demás siglas

Parte 2: Piezas básicas de la infraestructura

- ¿En la nube o no? Ese es el dilema...
    - Ventajas e inconvenientes de los entornos on premise
    - Ventajas e inconvenientes de los entornos cloud e híbridos
- Servidor Web
    - HTTP
    - DNS
    - HTTPS
- Balanceadores
- Servidores de aplicaciones
- Sistemas de bases de datos
- Sistemas de caché
- CDN
- API Gateway
- APM
- Monitorización de infraestructura
- Monitorización funcional
- Perfil SRE

Parte 3: Administración de sistemas unix

- Objetivo
- Contexto
- Virtualización de sistemas
- Diferentes distribuciones
- Montando el entorno
- Gestión de usuarios, permisos y accesos
    - Usuarios
    - Home del usuario
    - Grupos
    - Permisos

- Particiones en Linux
    - Tipos de particiones
    - Sistemas de Ficheros
    - Estructura de Directorios en Linux
    - Recomendaciones y buenas prácticas
    - Añadir un nuevo disco
    - Crear una partición
    - Mover a la nueva partición
    - Borrar una partición
    - Redimensionar una partición
- Editor Vi
    - Modos
    - Moverse con el cursor
    - Cambiar de Modo
    - Borrar texto
    - Copiar, cortar y pegar
    - Buscar y reemplazar
    - Manipulación de ficheros
- Procesos, servicios y tareas programadas
    - Herramientas para el manejo de procesos
    - Matar procesos
    - Listar los fichero abiertos por procesos
    - Arrancando una aplicación como servicio
    - Tareas programadas
- Bash Scripting
    - Variables
    - Sentencias condicionales
    - Operadores
    - Bucles
    - Manejo de ficheros
    - Funciones
    - Control de Errores
- Utilidades esenciales en UNIX
    - Comandos de red

    - Comandos entrada y salida
    - Comandos de filtrado
    - Comandos para buscar
    - Miscelanea

Parte 4: Docker

- Introducción a Docker
    - ¿Qué es Docker?
    - ¿Por qué se usa?
    - ¿Qué es un contenedor?
    - ¿Cómo funciona Docker?
    - Anatomía de contenedores
    - Instalación de Docker
- Trasteando con Docker
    - Arrancar mi primer contenedor
    - Arrancar, parar, borrar contenedores
    - Monitorizar los logs de un contenedor
    - Ejecutar una shell dentro de Docker
    - Configurando contenedores
- Construyendo imágenes a medida
    - ¿Qué es una imagen?
    - Creando imágenes
    - Compartiendo imagenes: subida y etiquetado
- Volúmenes de datos
    - Gestión de los volúmenes
- Información del Sistema en Docker
    - Consumo de recursos
    - Liberar recursos no utilizados

- Redes
    - Modelo de red en un contenedor
    - Cortafuegos
    - Implementaciones de red
    - Gestión de puertos
- Docker Compose
    - Arrancando un App Multi-Servicio
    - Construir imágenes con docker compose
    - Arrancar una aplicación con docker compose
    - Escalar un servicio

Parte 5: Ansible

- Introducción
    - Algo de historia
- Gestión de la configuración
    - Infraestructura como Código
    - Otras herramientas
- Arquitectura de Ansible
    - Preparando el entorno de pruebas
- Inventario
    - ¿Qué es?
    - Opciones de configuración
    - Hosts
    - Grupos
    - Variables
    - Múltiples inventarios
    - Inventario dinámico
- Playbooks
    - Variables y hechos
    - Plantillas con Jinja
- Roles
    - Estructura

        - Dependencias entre roles
        - Roles para diferentes plataformas
- Modulos
        - Custom modules
        - Depurando los módulos
        - AWS con Ansible
        - Docker con Ansible
- Protegiendo secretos en Ansible
        - Cifrar ficheros
        - Modificar fichero cifrado
        - Descifrar fichero
        - Cambiar la clave de cifrado
- Control de las tareas
        - Handlers
- Testing
- Debugging con Ansible
        - Comprobar sintaxis
        - Depuración de tareas
        - Logs
- Buenas prácticas
        - Organización de los archivos
        - Diferencias entre entornos
        - Lanzando tareas por lotes
        - Mencionar el estado
        - Uso de los roles
        - Diferentes sistemas operativos
        - Algunos consejos generales

Parte 6: Kubernetes

- Introducción
        - ¿Qué es Kubernetes?
        - Algo de historia
- Clúster de Kubernetes

        - Componentes
        - API de Kubernetes
        - Pasos para crear un clúster
- Contenedores
    - Imágenes
- Cargas de trabajo
    - Pods
    - Controladores
- Nodos
    - Estado de un nodo
    - Controlador de nodos
    - Registro de nodos
    - Comunicación Nodo-Maestro
- Almacenamiento
    - Volúmenes
    - Configuración de volúmenes
    - Límites
- Servicios, balanceo de carga y redes
    - Modelo de red de Kubernetes
    - Servicios de red
    - Enrutamiento
    - DNS
    - Ingress
    - Política de tráfico interno
    - Políticas de red
- Configuración
    - Consejos de configuración y buenas prácticas
    - Kubectl
    - ConfigMaps
    - Secrets

Bibliografía

Lecciones aprendidas con esta guía

Parte 1

Cultura de entrega de valor

Ideando el producto

Los negocios están continuamente buscando innovar para optimizar sus procesos y lograr la eficiencia operativa. A día de hoy la industria tiende a seguir un modelo ágil de desarrollo de software, donde se consiguen probar los requisitos de forma fácil y rápida. El objetivo es validarlos o desecharlos rápidamente, siguiendo el principio **“falla rápido, falla barato”**.

En nuevas iniciativas, se pretende construir un producto mínimo viable, más comúnmente conocido como MVP (Minimum Viable Product) para validar el conjunto de requisitos de negocio que se pretenden implementar. Este es un proceso iterativo e incremental, en el cual se entregan productos potencialmente utilizables aunque incompletos, construidos en ciclos cortos, en el que pueden surgir cambios o nuevos requisitos que se podrán aplicar durante las siguientes iteraciones (versiones del producto).



No siempre se tienen claros los detalles de las ideas o la profundidad de las mismas. En muchas ocasiones se parte de una semilla de idea, que va creciendo y cambiando de formas en la medida en que es validada con el cliente. Esta información no siempre puede preverse al principio.

Es importante que todo el equipo entienda que un cambio de prioridad o de la historia de usuario es posible y es necesario poder adaptarse a estos cambios siempre teniendo en cuenta el impacto que pueda ocasionar. Hay que evitar los extremos, ni poner trabas ni frustrarse ante dichos cambios, ni tampoco que éstos sean continuados.



En este sentido, DevOps promueve un conjunto de prácticas que combina el **desarrollo de software (dev)** con **operaciones y gestión** de la infraestructura sobre la que va a ir instalado el software **(ops)**. Todas estas partes participan juntas en el ciclo de vida de un producto o aplicación, y **cuanto más involucradas estén, más rápida y productiva será la entrega de valor**.

El flujo de desarrollo software

A continuación, se hace una aproximación de lo que puede ser el flujo de desarrollo de software de un equipo que no posee una cultura Devops y luego se introduce la implantación de prácticas que ayudan a mejorar en cada punto.

Suele constar a grandes rasgos de las siguientes fases:



Al negocio se le ocurre una idea para mejorar la entrega de valor a sus stakeholders o clientes.

Mediante un proceso de descubrimiento, análisis, refinamiento y priorización, que puede ser *ágil*, **al desarrollador le llega una necesidad o requisito, que podría estar formulada como una historia de usuario**. Esta puede estar escrita del siguiente modo: "*Como usuario quiero poder ver el detalle de cada animal que aparece en el listado*".

Ahora mismo el desarrollador se encuentra con la responsabilidad de **programar la funcionalidad** necesaria para que los usuarios finales puedan llevar a cabo esa acción.



Una vez **finalizada la programación** de la funcionalidad y testeada contra el entorno local del desarrollador **hay que subir los cambios a los distintos entornos**. Existirán al menos los siguientes entornos:



Esta subida suele hacerse de forma manual con riesgo a cometer errores humanos, resulta ser lenta y para nada es óptima. Imagina tener que hacer el proceso manualmente cada vez que haya un cambio, es decir, ejecutar los comandos necesarios de todas las herramientas en cada despliegue.

Estos comandos suelen requerir de un orden determinado y, al ser tan frecuente el proceso, lo mejor es automatizarlos. Así además evitamos errores por el factor humano. Por ello, como parte de la cultura DevOps, los cambios que se hagan en el desarrollo, al subir al repositorio de código, van a lanzar un proceso de **integración contínua** CI (continuous integration), que generalmente incluye la compilación del código, la ejecución de tests automáticos, la generación de artefactos y algunas otras fases como el análisis de calidad de código automático. **Si todo eso fuera correcto se podría ejecutar un proceso de despliegue continuo, (Continuous Deployment)** de tal forma que la nueva versión de la aplicación con los nuevos cambios incluídos estaría disponible para ser probada en un **entorno de integración**.

Integración continua

autentio

¿Qué es?

En inglés **Continuous Integration (CI)** es la práctica que tiene como objetivo integrar los cambios en el repositorio central de forma periódica a través de varios procesos automatizados donde cada versión generada se comprueba mediante pruebas y tests para detectar posibles errores de forma temprana.

¿EN QUÉ CONSISTE?

Durante el proceso de integración continua se pasan ciertas fases o tareas a ejecutar, como por ejemplo, instalar las dependencias necesarias, lanzar los tests, entre otros. En caso de que alguien del equipo decida hacer un Pull Request/Merge Request, comprobamos que dicho pipeline ha pasado correctamente y procedemos a integrar dichos cambios ya sea en la rama principal o en nuestra propia rama local en caso de necesitarlos.

¿Qué beneficios nos aporta?

- **Detección rápida de fallos** de forma continua.
- **Aumento de la productividad del equipo**
- **Automatización** y ejecución inmediata de procesos.
- **Monitorización** continua de las métricas de calidad del proyecto.



Debemos tener en cuenta:

- **Integrar los cambios** varias veces al día.
- La programación en equipo es un problema de **"divide, vencerás e integrarás"**.
- **La integración es un paso no predecible** que puede costar más que el propio desarrollo.
- **Integración síncrona**: cada desarrollador una vez completada la funcionalidad, sube sus cambios, espera a que se complete la construcción y se hayan pasado todas las pruebas sin ningún problema de regresión.
- **Integración asíncrona**: cada noche se hace un build diario en el que se construye la nueva versión del sistema. Si se producen errores se notifica con alertas de emails.
- **El sistema resultante debe ser un sistema listo para lanzarse**

Despliegue continuo

autentio

¿Qué es?

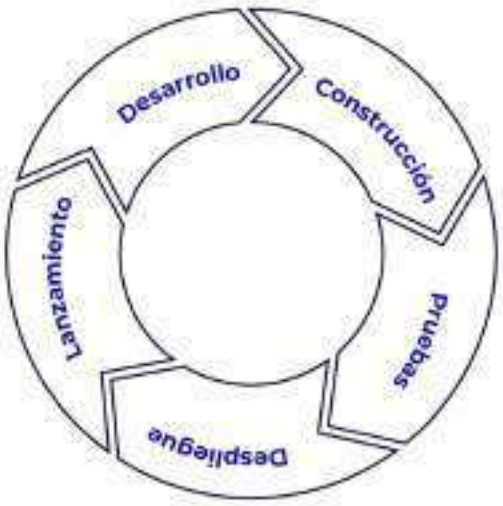
En inglés **Continuous Deployment (CD),** es una práctica que tiene como objetivo proporcionar una manera ágil, fiable y **automática** de poder entregar o desplegar los nuevos cambios en el entorno específico, normalmente producción. Suele utilizarse junto con la integración continua.

¿EN QUÉ CONSISTE?

El despliegue continuo se basa en automatizar todo el proceso de despliegue de la aplicación en cualquiera de los entornos disponibles, ya sea sólo en algunos de ellos o en todos, todo ello, sin que haya **ninguna intervención humana** en el procedimiento.

El objetivo es hacer **despliegues predecibles, automáticos y rutinarios** en cualquier momento, ya sea de un sistema distribuido a gran escala, un entorno de producción complejo, un sistema integrado o una aplicación.

Dependiendo de cada proyecto o propósito de negocio de la empresa, el despliegue estará configurado para hacerse de forma semanal, quincenal o cada 2 o 3 días, aunque el objetivo es hacerlo de manera constante y en periodos cortos para obtener feedback rápido del cliente.

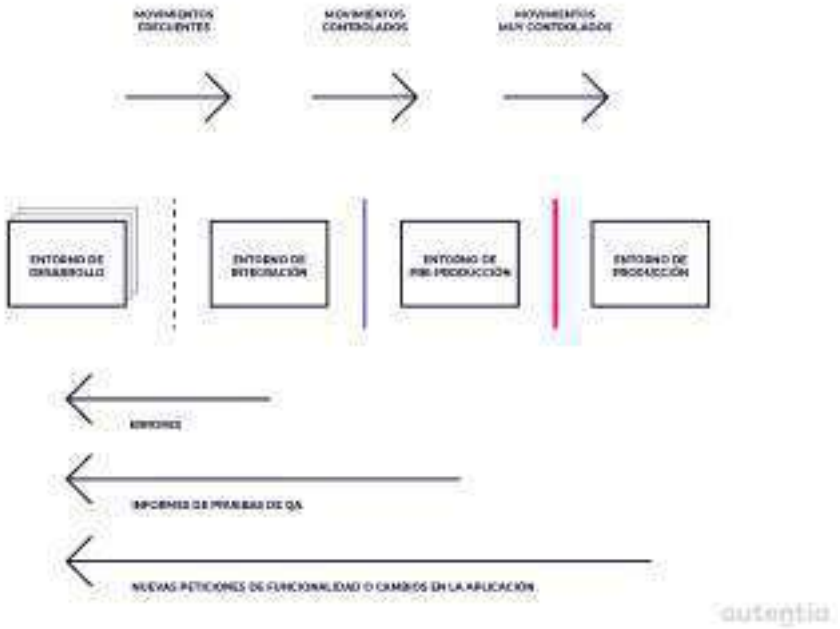


El **entorno de integración** es usado en su mayoría por los desarrolladores para verificar que la funcionalidad entregada cumple con los requisitos y no tiene errores. Es tan importante aportar nueva funcionalidad como no romper las que ya se habían entregado.

Tenemos que ser conscientes de que en las empresas suele haber muchos equipos de desarrollo que necesitan usar el entorno de integración. **No dejar al libre albedrío el acceso y modificaciones de este entorno permite reducir errores humanos** (como la ejecución de comandos incorrectos, la subida de versiones sin aprobación de los tests o, incluso, el olvido de la ejecución de alguno de los pasos del proceso) que, bien por inexperiencia o falta de conocimiento, pueden ocurrir. En este sentido, **lo ideal es que los cambios sean automatizados**.

Si las pruebas han ido bien y hemos verificado que la nueva funcionalidad es correcta en el entorno de integración, el siguiente paso sería promocionar al siguiente entorno: el **entorno de preproducción**. Este debe ser aún más estable que el anterior, ya que puede haber terceros utilizándolo para integrarse con nosotros, QAs probando historias...

Si todo es correcto terminaría el ciclo de desarrollo poniendo esta funcionalidad en el **entorno de producción**. Si preproducción tenía que ser un entorno estable, el entorno de producción muchísimo más. Aquí es vital la automatización de procesos.



Empezando con el desarrollo

Una vez tenemos definido lo que tenemos que hacer, es el turno del desarrollador de software. Este va a hacer uso de una serie de herramientas para hacer su trabajo.

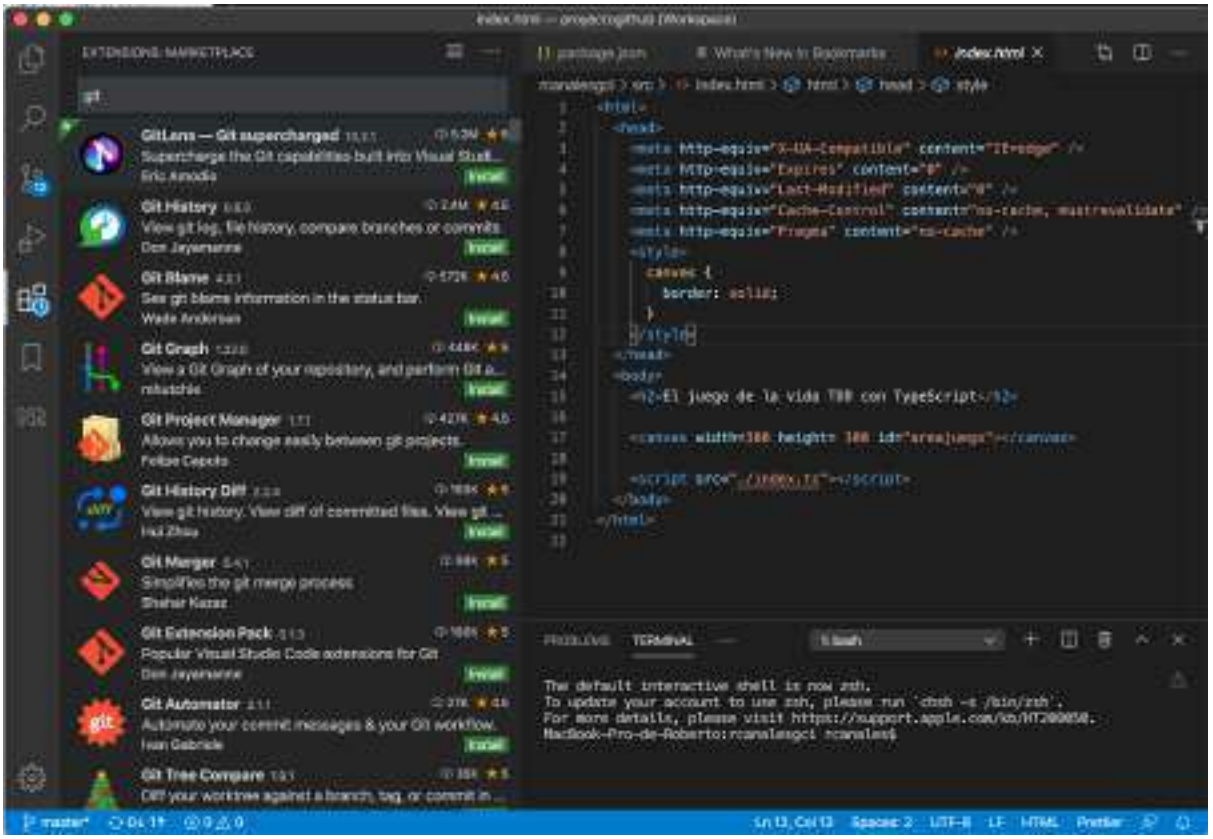
En un mundo cada vez más interconectado, los profesionales requieren autonomía, por lo que tendrán estas herramientas instaladas en su ordenador (probablemente un portátil con grandes capacidades), configuradas para poder trabajar con los proyectos. Al conjunto de estas herramientas se las conoce como "**entorno local de un desarrollador**". Vamos a ver algunas de las más comunes.

El uso del IDE

Un **IDE** (Integrated Development Environment) es una interfaz de desarrollo, una herramienta que nos permite realizar desde las tareas más básicas, como codificar, hasta otras más avanzadas o accesorias, como hacer cambios en múltiples ficheros al mismo tiempo.

A menudo se pueden extender instalando plugins, que son extensiones que han desarrollado terceros y que añaden funcionalidad extra o utilidades que van a simplificar nuestras tareas diarias. Estos plugins nos permiten, por ejemplo, ver el historial de cambios que ha tenido un fichero y quienes lo han modificado. Para ello harán uso de un sistema distribuido de control de versiones como Git (que veremos más adelante). Hay muchos IDEs distintos que ofrecen características y opciones diferentes y pueden estar centrados en distintos lenguajes de programación. Entre los más populares, encontramos Eclipse, IntelliJ IDEA o Visual Studio Code.

Hemos visto que podemos llegar a trabajar junto con decenas o incluso centenas de programadores, pero ¿cómo se coordina todo esto?



Ejemplo de IDE: Visual Studio Code con plugins disponibles.

Git

Git es, como dice su propia página web, un **sistema de control de versiones distribuido**, de código abierto y gratuito, es decir, donde vamos a guardar el código fuente de nuestras aplicaciones y todos los cambios que se hagan sobre el mismo. La palabra clave es "distribuido". En el caso de Subversion, CVS o similares hay un repositorio central con el cual se sincroniza todo el mundo. Este repositorio central está situado en una máquina concreta y es el repositorio que contiene el histórico, etiquetas, ramas, etc. En los sistemas de control de versiones distribuidos, como es el caso de Git, esta idea de repositorio central no existe, está distribuido entre los participantes; cada participante tiene en su local el histórico, etiquetas y ramas. La gran ventaja de esto es que no necesitas estar conectado a la red para hacer cualquier operación contra el repositorio, por lo que el trabajo es mucho más rápido y tiene menos dependencias. Ante esta idea hay algunas preguntas comunes que suelen asaltar nuestra cabeza:

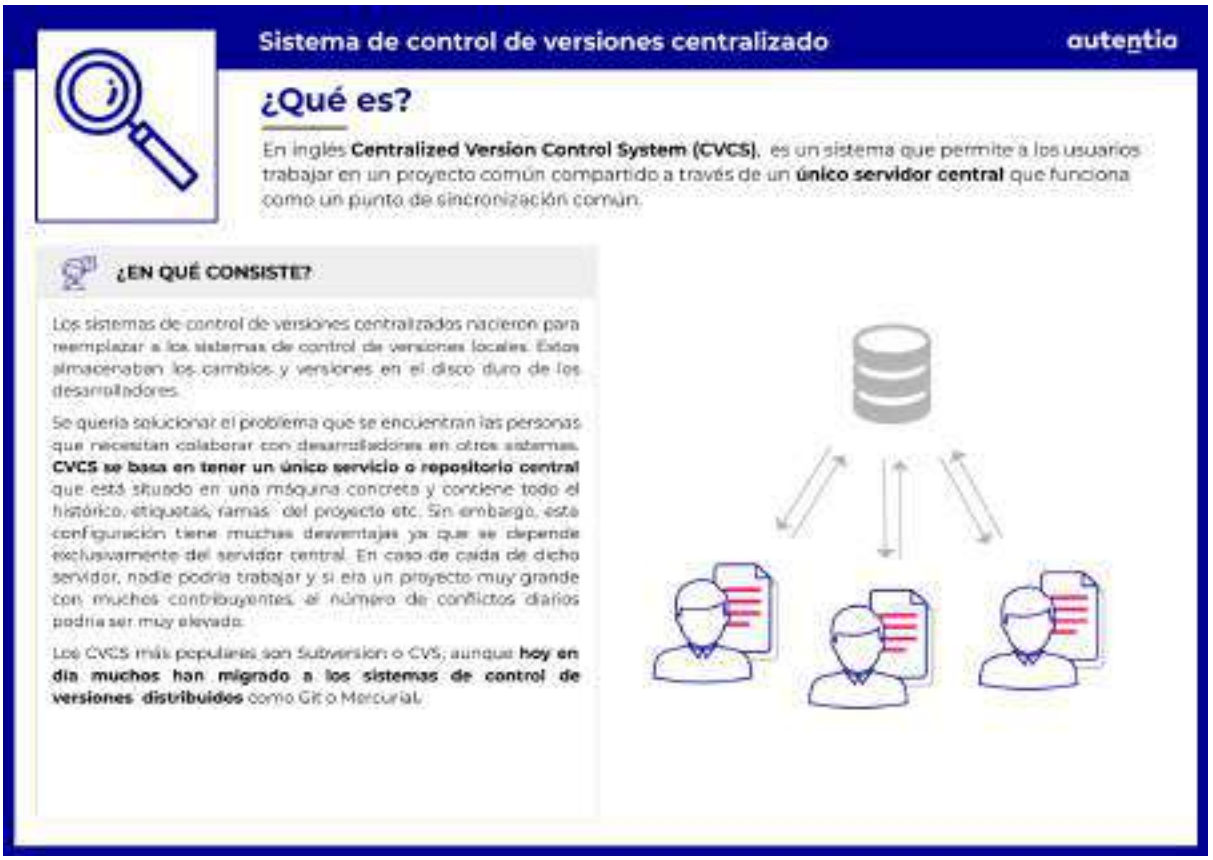
- **¿Si tenemos todo el repositorio en local, no ocupará mucho espacio?**

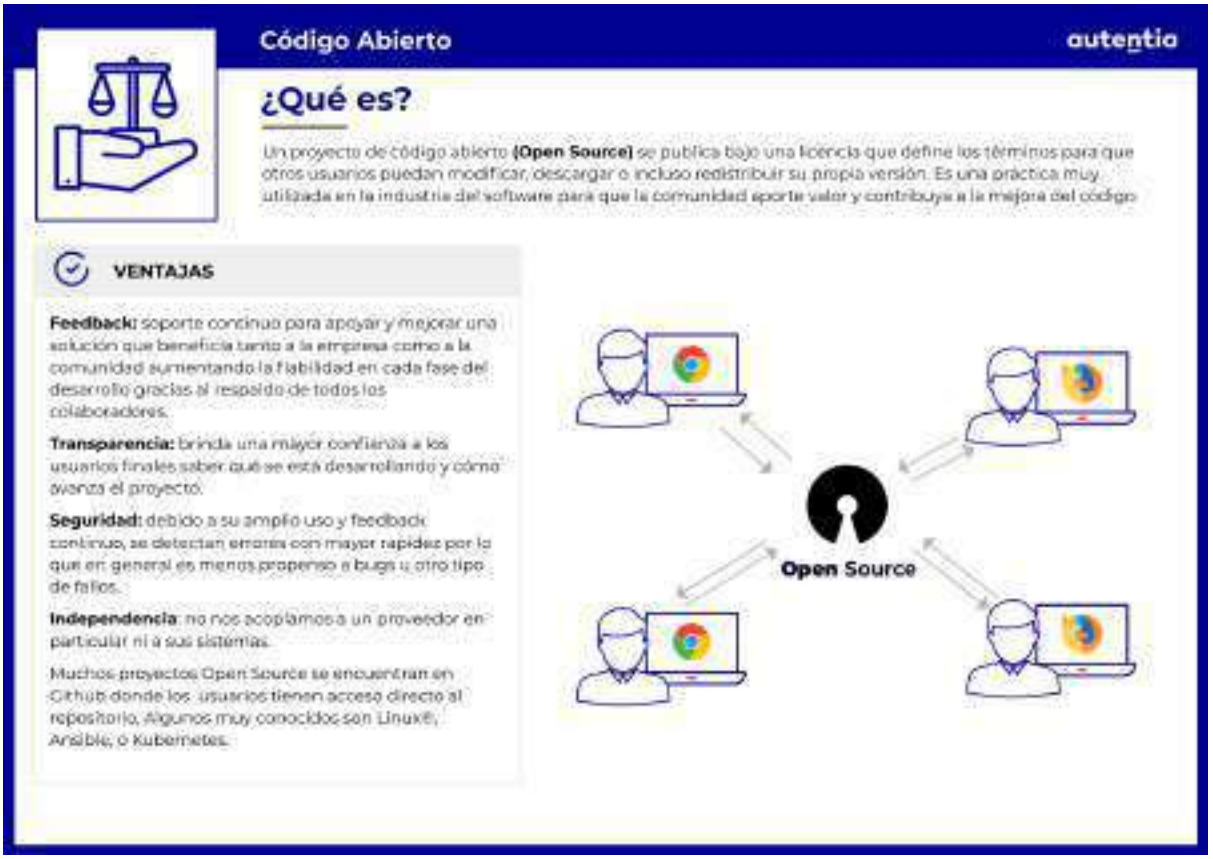
  Lo normal es que no, porque al ser un repositorio distribuido, solamente tendremos las partes que nos interesan.
  Habitualmente, si tenemos un repositorio central, tendremos muchos proyectos, ramas, etiquetas, etc. Al tratarse de un repositorio distribuido, solo tendremos en local la parte con la que estamos trabajando (la rama de la que hemos partido).

- **¿Si todo el mundo trabaja en su local, no puede resultar en que distintas ramas diverjan?**

  Efectivamente, esto puede ocurrir y es natural. En un desarrollo tipo open-source no hay mucho problema, es lo habitual y constantemente están saliendo nuevos "forks" (desarrollos paralelos y distintos del original) de los productos. En un desarrollo "interno" tampoco hay problema si ocurre esto, porque luego vamos a acabar haciendo "merge" (fusionando los cambios que tenemos en local con los que hay en el repositorio, para que tanto la versión local como la del repositorio sean iguales). Al final, todo depende de nuestro proceso de desarrollo, no tanto de la herramienta que usemos; es normal que los desarrolladores abran nuevas ramas constantemente para desarrollar partes del sistema y que luego hagan merge para unir estos cambios sobre una rama principal.

Si quieres saber más sobre esto, en el artículo **Git y cómo trabajar con un repositorio de código distribuido** podrás encontrar más información.

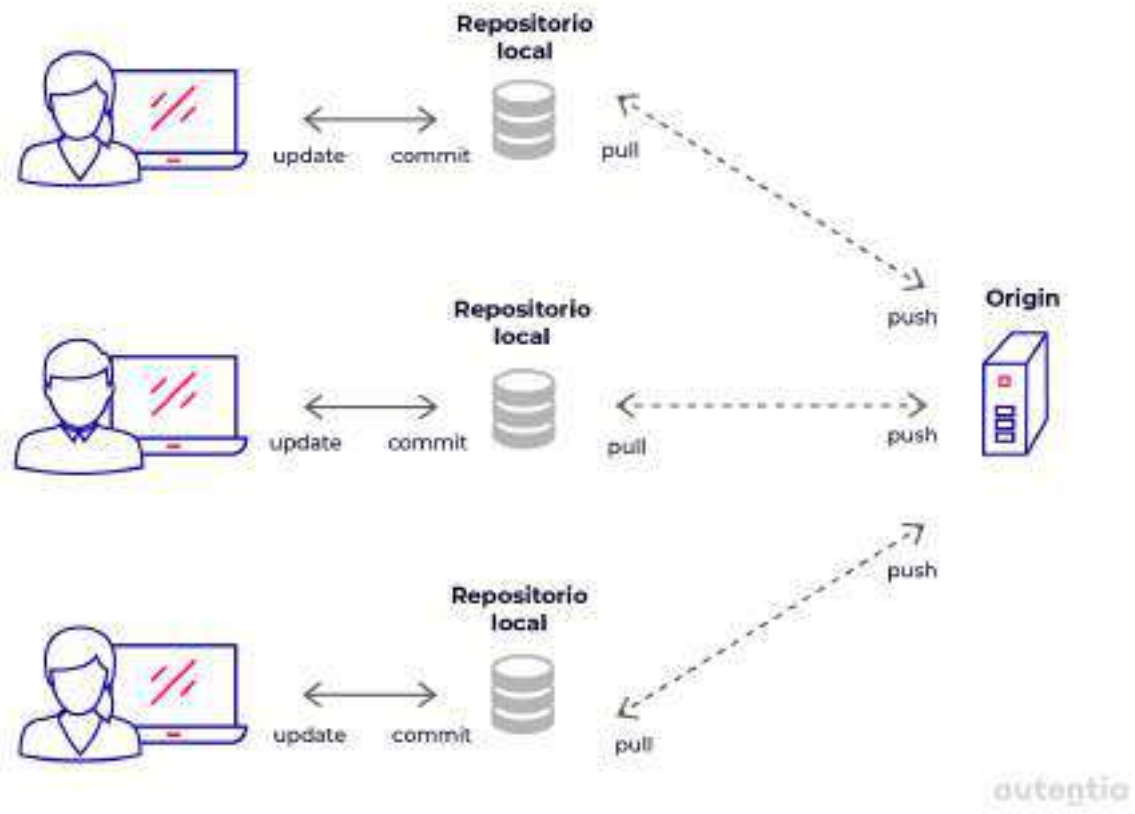




Git Flow

Git Flow surge de un [post de Vincent Driessen en 2010](#) donde explica un modelo de desarrollo basado en las ramas de git utilizando una serie de convenciones para el nombrado de las ramas y definiendo un uso específico de las mismas.

Aunque el enfoque de Git es descentralizado, se necesita un **repositorio central** que contenga los cambios hechos por los componentes del equipo. A este repositorio se le llama “origin”. Cada desarrollador hace “pull" (traerse los cambios del origen a tu carpeta local) y “push” (subir los cambios de la carpeta local al repositorio origen) al origen. Además de las relaciones centralizadas, cada desarrollador puede hacer pull de cambios de otros compañeros para formar subequipos. Esto es útil cuando trabajas con dos o más desarrolladores en una gran característica nueva, antes de llevar el trabajo en progreso al “origin” prematuramente y causar que haya un desarrollo incompleto.



El autor del post coloca una nota que aclara que este modelo no aplica a todos los equipos y que **no se debe forzar su adopción**. No es un dogma que haya que seguir siempre en todo tipo de proyectos. Este modelo tiene sentido en un proyecto donde haya que tener un fuerte control de las distintas versiones del proyecto, manteniendo y dando soporte a varias versiones a la vez. En cambio, si el proyecto no requiere de mantenimiento de distintas versiones a la vez, de tener que volver atrás a una versión antigua, puede que no tenga tanto sentido ceñirse a git flow.

Ramas principales

El repositorio central tiene dos ramas principales, paralelas, que nunca se borran y sobre las que todo el sistema de ramas de soporte se va a apoyar:

- **Master**: rama que contiene el código listo para subir a producción o hacer una nueva versión; este código será el del commit al que apunte HEAD (el último commit de la rama).
- **Develop**: rama que siempre refleja el estado del desarrollo con los últimos cambios listos para ser entregados en la próxima versión; este código será el del commit al que apunte HEAD (el último commit de la rama). Esta rama puede tener otros nombres como, por

ejemplo, rama de integración.

Cuando el código fuente en la rama de desarrollo alcanza un punto estable y está listo para ser lanzado en forma de nueva versión, todos los cambios deben fusionarse nuevamente en la rama master y luego etiquetarse con un número de versión.

Por lo tanto, cada vez que los cambios se fusionan en la rama master, por definición, se genera una nueva versión de producción. Podríamos compilar y desplegar automáticamente el software en producción cada vez que haya un commit en master.

Además de las ramas master y develop, el modelo de desarrollo utiliza una variedad de ramas de soporte para ayudar al desarrollo paralelo entre los miembros del equipo, facilitar el seguimiento de las características, prepararse para los lanzamientos de producción y ayudar a solucionar rápidamente los problemas de producción en vivo. A diferencia de las ramas principales, estas ramas siempre tienen un tiempo de vida limitado, ya que una vez cumplido su propósito, se eliminan.

Los diferentes tipos de ramas que podemos usar son: feature, release y hotfix.

Cada una de estas ramas tiene un propósito específico y están sujetas a reglas estrictas sobre qué ramas pueden ser su rama de origen y a qué ramas pueden hacer merge.

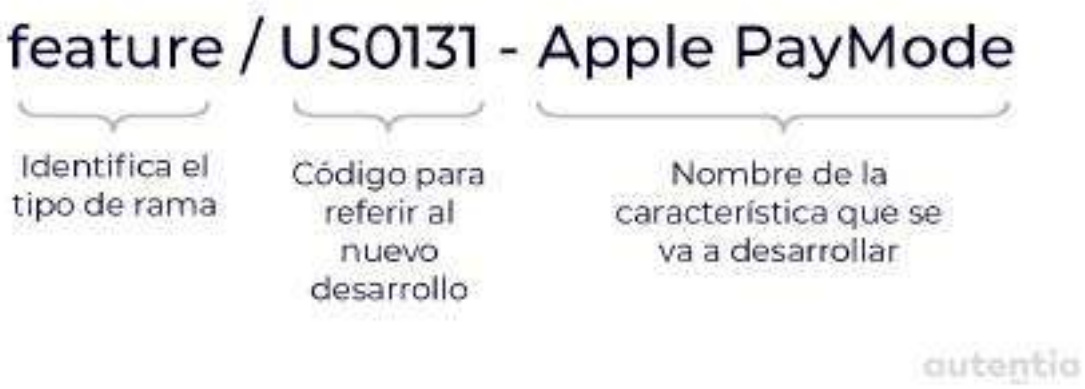
Estas ramas no son especiales desde una perspectiva técnica, son ramas normales de git y esta clasificación se hace por el tipo de uso que pretendemos darle al código dentro de la rama:

Ramas Feature:

Utilizadas **para desarrollar nuevas funcionalidades** para las próximas versiones. Al comenzar el desarrollo de una funcionalidad, la versión de destino en la que se incorporará puede ser desconocida en ese momento. La esencia de una rama feature es que existe mientras la funcionalidad esté en desarrollo, pero **finalmente se fusionará de nuevo en la rama develop** (para agregar definitivamente la nueva funcionalidad a la próxima versión) o se descartará (en caso de que la característica resulte ser un experimento decepcionante) borrando la rama.

Esta rama **se va a crear partiendo de la rama develop y, al terminar, se va a fusionar (merge) con la rama develop**. La convención de nombres de estas

ramas suele ser feature/(nombre de la característica a desarrollar). Si se está usando un sistema para manejar las tareas del desarrollo y estas tareas tienen un número o identificador, el nombre de estas ramas suele ser feature/(identificador)-(nombre de la característica a desarrollar).



Ramas Release:

Apoyan la preparación de una nueva versión de producción. Además, **permiten pequeñas correcciones de errores** y preparan metadatos para un lanzamiento (número de versión, fechas de compilación, etc.). Al hacer este trabajo en una rama release, la rama de desarrollo puede ser usada para recibir características para la próxima gran versión sin crear problemas al cruzar trabajo entre dos versiones.

El momento clave para crear una nueva rama release desde la rama de desarrollo es cuando la rama desarrollo (casi) refleja el estado deseado de la nueva versión. Antes de crear la rama release desde la de develop, todas las funcionalidades que tienen que ir en la nueva versión tienen que ser primero fusionadas con la rama develop; luego ya se puede crear la rama release. Las ramas feature que contengan funcionalidades no pertenecientes a la versión que se va a lanzar a la rama release, no deben ser fusionadas con la rama develop.

Es exactamente en la creación de una rama release cuando se le asigna un número de versión, no antes (de acuerdo a las reglas de versionado del proyecto). Hasta ese momento, la rama de desarrollo contiene cambios para la "próxima versión", pero no está claro si esa "próxima versión" se convertirá finalmente en la 0.3 o 1.0 hasta que se cree la rama. Aún así, no se va a sacar versión del proyecto hasta que se haga el merge con master y, por lo tanto, los eventuales bugs que pueda tener esta rama estén

resueltos.



Esta rama se va a crear partiendo de la rama develop y al terminar se va a fusionar (merge) con las ramas develop y master. La convención de nombre de estas ramas suele ser release/(número de versión).

Ramas Hotfix:

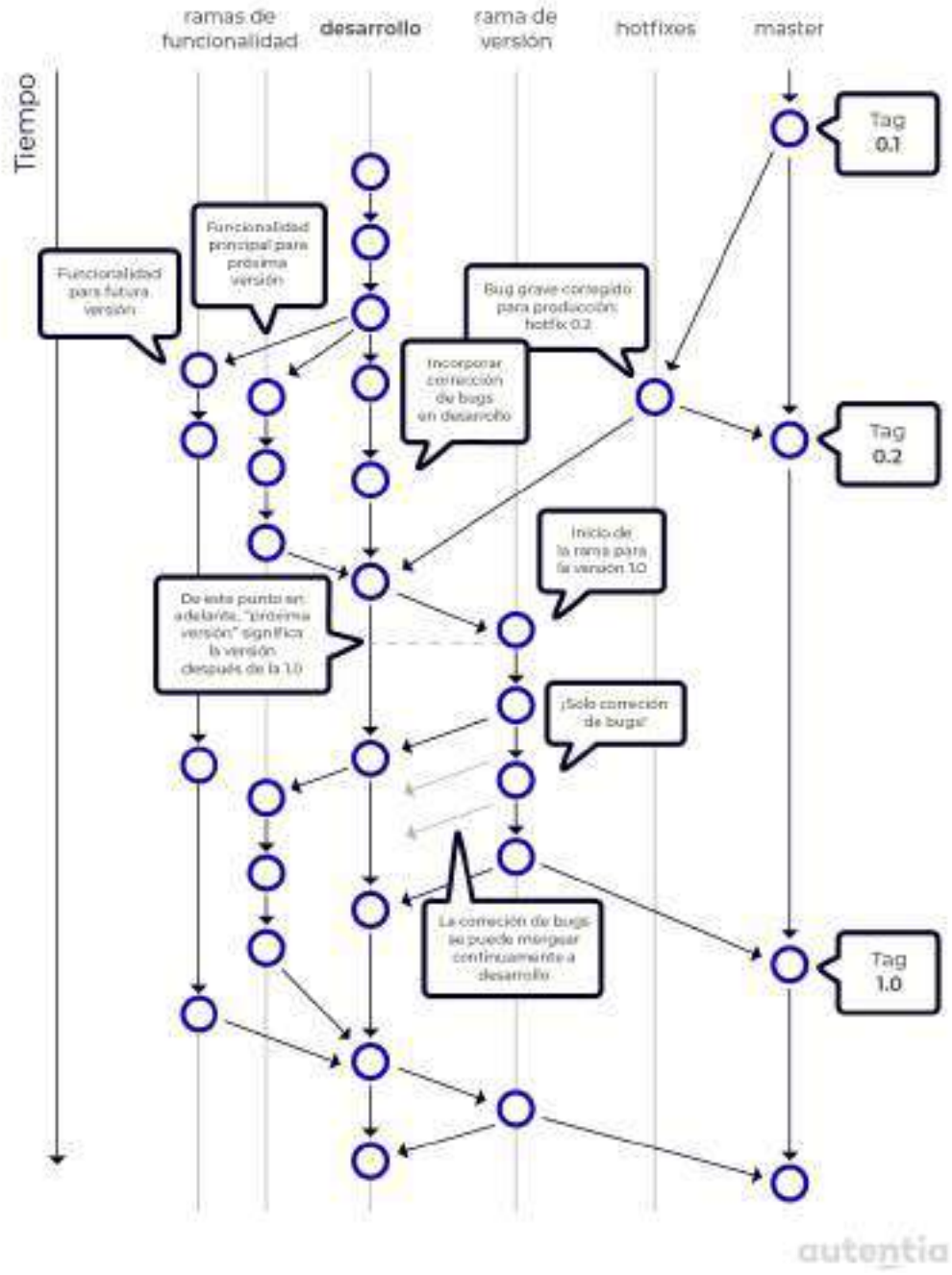
Muy parecidas a las ramas release, ya que también están destinadas a prepararse para un nuevo despliegue en producción. Surgen de la necesidad de actuar inmediatamente sobre un estado no deseado (un bug o error) de una versión que se está usando en producción. **Cuando se descubre un error crítico** en una versión de producción **debe resolverse de inmediato**, una rama hotfix se puede crear partiendo de la rama master que esté marcada (con un tag) con el número de la versión desplegada en producción.

La esencia de esta rama es que el trabajo de los miembros del equipo (en la rama de desarrollo) puede continuar, mientras que otra persona está preparando una solución rápida para producción.

Esta rama se va a crear partiendo de la rama master y al terminar se va a

fusionar (merge) con las ramas develop y master. La convención de nombre de estas ramas suele ser hotfix/(número de versión, incrementado sobre la versión que hay en producción).

Lo importante en la convención de nombres de las ramas auxiliares es indicar el tipo de rama, poner la barra como separador y darle un nombre claro para que cualquiera que vea la rama sepa el trabajo que se está realizando en ella.



Documentación con más explicaciones y comandos para poner en práctica Git Flow.

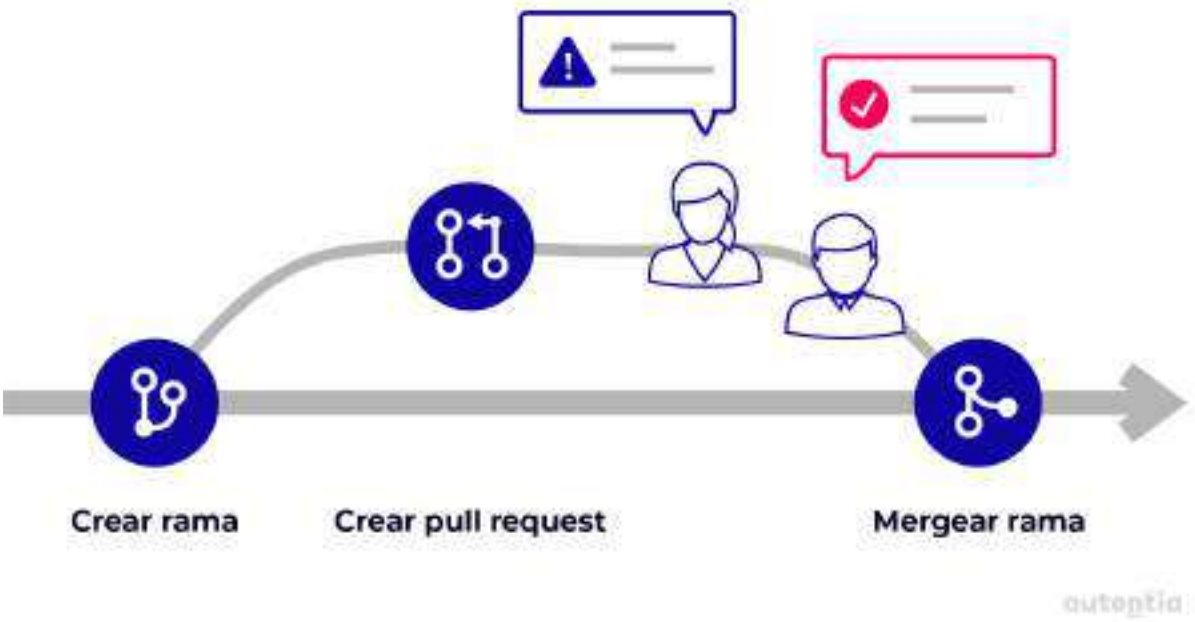
GitHub Flow

Es un flujo definido por GitHub para **estandarizar** la forma en la que todas las **contribuciones** en los proyectos comunitarios de GitHub tienen que hacerse. Está pensado para equipos y proyectos en los que se realizan **despliegues regularmente**. Este flujo también puede aplicarse fuera de GitHub como sucede con GitFlow, ya que sigue el mismo patrón de centralizado y distribuido.

En este modelo solo hay una regla: todo **el código que hay en la rama master tiene que poder desplegarse en cualquier momento.** Por este motivo, siempre que se quiera hacer un cambio, hay que crear una rama a partir de la rama master y darle un nombre descriptivo. En esta nueva rama creada se pueden hacer tantos commits como se quiera y experimentar, sabiendo que la rama no será fusionada (merge) con master hasta que alguien la revise y la apruebe. El mensaje incluido en cada commit tiene que ser descriptivo y detallado para que la persona o personas que revisen los cambios hechos en la rama puedan entender qué se van a encontrar al leer el histórico de commits.

Una vez terminado el trabajo en la rama y pasados todos los tests, se tiene que crear una **pull request** (petición de fusión) de tu rama con la rama master. Una pull request sirve para hacer "**code review**" (revisión de código), que es una técnica de mejora de calidad de código y transmisión efectiva del conocimiento, en el que cada funcionalidad o pieza de código que tenga cierta entidad pasa por un proceso de revisión donde las personas encargadas, que tendrán que ser añadidas para revisar esa pull request, comprueban el código y sugieren mejoras en el mismo a través de comentarios en la pull request. Una vez se resuelvan los comentarios o puntos pendientes y aprueben la pull request, esta se podrá fusionar con master.

Cuando los cambios han sido fusionados, se harán tests de integración para comprobar que funciona correctamente con el código que ya había y se desplegará una nueva versión en producción. Si en algún momento hay algún problema, se puede hacer "**rollback**" (volver a una versión anterior que funciona) usando la versión anterior de la rama master.



Trunk based development

Esta forma de trabajar apuesta **por no usar ramas**, para así evitar los merges grandes que pueden surgir de no hacer merge durante días o semanas. Para que esta técnica funcione los equipos deben ser maduros, ya que no están haciendo commits en una rama de código que nadie ve, sino en la rama principal que usan todos los desarrolladores. **Romper la build** y subir algo que no pase los tests, o que no compile, **puede parar a decenas de personas**, por ello el equipo debe ser experimentado. Es posible, además, que se suban cambios a la rama principal que no están acabados; es código que va a ir a producción y, sin embargo, no debe ejecutarse. Para esto **se usan las feature toggles**, de las que puedes leer más en el siguiente enlace.

La **ventaja principal de este enfoque es que las integraciones de los cambios en la rama principal aunque se hacen más frecuentemente, éstas son más sencillas de resolver ya que existen menos posibilidad de conflicto al ser más acotadas y por lo tanto se reduce el número de errores que estos conllevan**. Se puede encontrar más información en el artículo Trunk-Based development: pusheando a master.

Otro artículo que se podría considerar de obligada lectura para la profundización en los mecanismos o patrones de integración de ramas es: Patterns for Managing Source Code Branches

Entorno local

No todos los lenguajes son compilados, ciertos lenguajes como Java, Kotlin, C y otros, requieren el uso de un compilador. En el caso de algunos lenguajes, este viene incluído en el **IDE**. Pero normalmente queremos tener el compilador instalado y configurado por fuera del IDE para poder compilar o ejecutar la batería de tests y verificar que los cambios realizados recientemente no hayan introducido nuevos errores o bugs al código ya existente.

Otras herramientas que encontramos en un **entorno local de desarrollo** son aquellas que nos permiten ejecutar el código en un entorno similar al productivo con la principal finalidad de validar que la aplicación cumple con los requisitos funcionales mínimos y que no tiene fallos antes de publicar dichos cambios en los repositorios y/o promoverlos a otros ambientes. Entre estas herramientas podemos encontrar **servidores web** como Apache o Nginx, **servidores de aplicaciones** como Apache Tomcat, JBoss Wildfly, Jetty, Liberty, que incluso pueden ser ejecutados localmente dentro de contenedores Docker, o servidores de desarrollo incluidos en la tecnología que estemos usando como, por ejemplo, Angular CLI. Otras herramientas que se suelen utilizar para estas pruebas en el desarrollo mobile son los **emuladores de sistemas operativos móviles**, o incluso **un móvil conectado al ordenador** y al entorno de desarrollo, para ejecutar y verificar la aplicación en un entorno lo más cercano al real. Hablaremos de la función de estas herramientas más adelante.

Muchas aplicaciones utilizan **bases de datos** para el almacenamiento de información o datos del negocio. Para poder desarrollar en un entorno similar al productivo, muchas veces es necesario utilizar bases de datos de forma local, ya sean ejecutadas en memoria dentro de la aplicación cómo podrían ser H2 o SQLite, o instaladas localmente (o bien usando algún mecanismo de virtualización ligero como Docker), como PostgreSQL, MySQL o MariaDB entre otras. Además, estas bases de datos se suelen utilizar para tener algunos **datos de prueba**, y probar modificaciones sobre el esquema o los datos **sin impactar al resto de entornos que están siendo usados por muchas personas.**

Base de Datos

autentia

¿Qué es?

Una base de datos es una especie de "almacén" que nos permite guardar grandes cantidades de información para su posterior **recuperación, análisis y/o transmisión**. Podemos encontrar diversos tipos de bases de datos y se clasifican de acuerdo a las necesidades que busquen solucionar en dos grandes bloques.

TIPOS

Concepto	Definición	Ejemplos
Relacionales	Los datos se relacionan entre ellos a través de identificadores en tablas. El lenguaje predominante es SQL (Structured Query Language). Aportan a los sistemas mayor robustez y ser menos vulnerables ante fallos gracias a su **atomicidad, consistencia, aislamiento y durabilidad (ACID)**	MySQL, PostgreSQL, Oracle, SQLite.
No relacionales	No siguen un patrón fijo como estructura de almacenamiento por lo que **su uso es muy común cuando no se tiene un esquema exacto de lo que se va a almacenar.** Se pueden encontrar bases de datos de gráficos, orientada a documentos, de columnas (Wide column store) o de claves-valor. Algunas desventajas es que no todas contemplan la atomicidad ni la integridad de los datos.	MongoDB, Redis, Elasticsearch, Cassandra, DynamoDB.

No Relacionales	Definición	Ejemplos
Clave-Valor	Cada elemento en la base de datos se almacena como un par (clave-valor) donde la clave sirve como un identificador único. Tanto la clave como valor pueden ser cualquier tipo de primitivo, como objetos compuestos.	DynamoDB, Redis, Riak, Voldemort
Columnas (Wide column stores)	También conocidas como familia de columnas o base de datos columnar. **Permiten manejar un gran volumen de datos** y mezclan conceptos de las bases de datos relacionales con una base de datos clave-valor. **Almacenan tablas de datos como secciones de columnas** en lugar de filas de datos y en cada sección se puede encontrar elementos con clave-valor	Cassandra /Hbase, Microsoft Azure Cosmos

Base de Datos

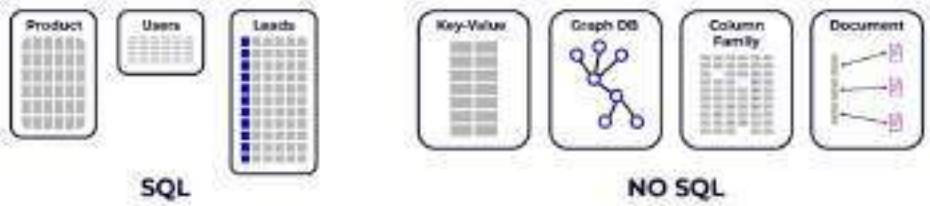
autentia

¿Qué es?

Una base de datos es una especie de "almacén" que nos permite guardar grandes cantidades de información para su posterior **recuperación, análisis y/o transmisión**. Podemos encontrar diversos tipos de bases de datos y se clasifican de acuerdo a las necesidades que busquen solucionar en dos grandes bloques.

TIPOS

No Relacionales	Definición	Ejemplo
Documentales	**Los datos se almacenan y se consultan como documentos tipo JSON**. Los documentos pueden contener muchos pares diferentes clave-valor, o incluso documentos anidados. Son más flexibles al cambio ya que si el modelo de datos necesita cambiar, solo se deben actualizar los documentos afectados y no todo el esquema.	MongoDB
De Grafos	**Usan nodos para almacenar entidades de datos y aristas para almacenar las relaciones entre nodos**. El valor de estas bases de datos se obtiene de las relaciones entre nodos y no hay un límite para el número de relaciones que un nodo pueda tener. Un borde siempre tiene un nodo de inicio, un nodo final, un tipo y una dirección.	Neo4J, HyperGraphDB



Adicionalmente, podemos ejecutar en nuestro entorno local herramientas como **Sonar**, para realizar inspecciones preliminares del código y verificar su calidad. Estas verificaciones se pueden lanzar también mediante plugins integrados en el IDE.

En local funciona, y ahora qué...

Una vez que nuestro código compila y cumple los criterios de aceptación funcional solicitados ya es hora de subir nuestros cambios, dejarlos disponibles al resto de los desarrolladores y publicarlos en los entornos de prueba.

Para esto hay que realizar varios pasos, comenzando por hacer un push al repositorio GIT. Según la forma de trabajo del proyecto, esto puede requerir hacer un Pull Request para poder integrar nuestros cambios en la rama principal del proyecto.

Luego, habrá que compilar nuestro código y generar el entregable correspondiente, ya sea una librería o una aplicación. Dichos entregables se recomienda sean publicados en un repositorio de software, como Nexus o NPM para desplegarlos posteriormente en un ambiente de pruebas, ya sea subiendo mediante FTP o mediante herramientas de despliegue como las incluidas en los servidores de aplicaciones (aplicación web o despliegue remoto). Si hemos usado la opción del FTP, seguramente también tendremos que reiniciar algún servidor para que se apliquen las actualizaciones.

Por lo general, este ambiente de pruebas es conocido como **entorno de integración**.



Entorno de integración

Este entorno seguramente será **distinto** al nuestro de **desarrollo local**, ya que lo **ideal** es que sea lo más **parecido** posible al **entorno de producción**. Es deseable que a nivel infraestructura, este entorno utilice la misma versión de base de datos, el mismo sistema operativo, la misma versión de servidor web y / o de aplicaciones, sistemas de virtualización y orquestación, etc. Este entorno y el entorno de producción no van a usar la misma base de datos, cada uno tendrá la suya. Pero será el mismo software y en la misma versión para asegurarnos de que lo que funciona en un entorno, funcione en el otro. Seguramente, y por un tema de costes, este entorno no disponga de los mismos recursos de hardware que el entorno productivo, y por un tema de seguridad y privacidad disponga de datos de prueba distintos a los existentes en producción. La configuración y parametrización de este entorno será muy similar a la de los demás entornos pre-productivos e incluso al de producción, pero siempre pueden existir particularidades.

Todas estas tareas manuales, además de suponer mucho más trabajo, son un posible punto de generación o introducción de problemas o errores. Aquí es donde la filosofía DevOps viene al rescate, haciendo uso de conceptos como la integración y la entrega contínua (CI/CD), la infraestructura y la configuración cómo código (IaC, Infrastructure as Code, y CaC, Configuration as Code), y otros conceptos. Todo va encaminado a lo mismo: que el desarrollador se preocupe solamente de desarrollar, y tareas como la configuración de entornos, subida de artefactos, el despliegue de los mismos se realicen de forma automática y predecible.

Integración Continua (CI)

Supongamos que el equipo finalizó el desarrollo de un conjunto de funcionalidades que deben ser incluidas en la próxima release del producto. Se acerca la fecha de liberación de esta release, por lo que debemos fusionar (o mergear) toda esta nueva funcionalidad en la rama master del proyecto. Hacer merge de todos estos cambios en la rama principal no solo es una tarea ardua y complicada, sino que puede hacer que surjan problemas o errores inesperados y, lo que es peor, que algunos de estos errores o bugs se "filtren" a la rama master y, de esta forma, al entorno productivo. Cuanta más gente hay trabajando en el proyecto, mayor es la probabilidad de que esto ocurra.

Esto es lo que se suele conocer como "**Merge Day**", un día entero o gran parte del día donde se paran todos los desarrollos y todo se centra en conseguir que la fusión de todos los cambios en la rama master salga bien. Siempre hay conflictos en los merges y si no se tiene un buen conocimiento de git y del trabajo hecho, seguro que alguien pisa el cambio de otro y cuando se prueba la integración todo falla y nadie sabe el porqué. En ese momento comienza una búsqueda para encontrar el fallo y al culpable, lo que crea muchas tensiones en el equipo. Además de esto, imagina que esto ocurre un viernes y te tienes que quedar horas extras, pues no puedes irte de fin de semana teniendo fallos en el sistema.

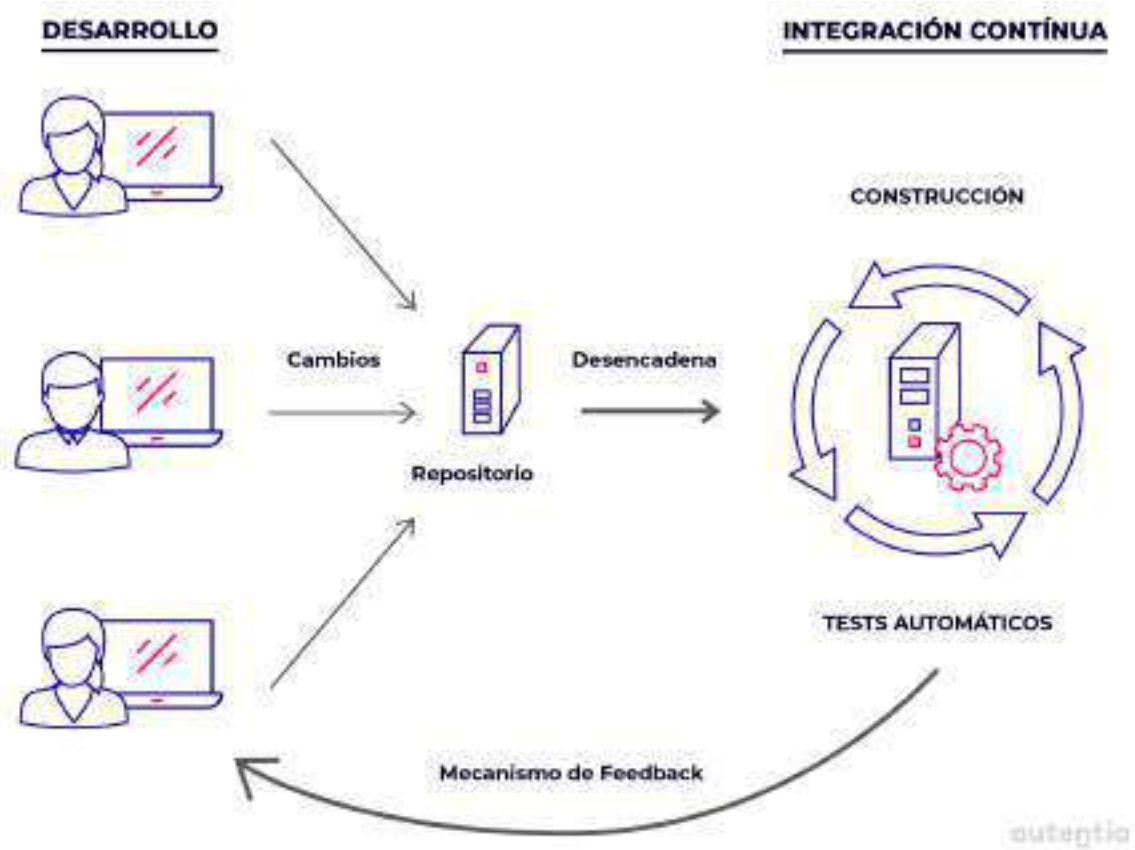
La Integración Continua (CI) surge para ayudar con esta parte del proceso. Tal cómo lo explica Martin Fowler en su artículo, ***"La integración contínua es una práctica del desarrollo de software en la que los miembros de un equipo integran su trabajo con frecuencia. Usualmente cada persona integra sus cambios al menos diariamente - lo que lleva a múltiples integraciones por día. Cada integración se verifica mediante una compilación automatizada, que incluye las pruebas y tests, para detectar errores de integración lo más rápido posible."***

Esta práctica permite, entre otras ventajas, **mejorar la productividad del desarrollo**, **reducir la cantidad de problemas** de integración, **detectar errores** y fallos **de manera temprana**, realizar actualizaciones y **entregar valor con mayor rapidez**.

Pero no hay que olvidarse de la otra gran ventaja que aporta esta práctica: **la transparencia del proceso**. Como todos trabajamos utilizando el mismo repositorio de código y la información del servidor de integración es

pública, para todo el equipo e incluso a veces para el cliente, se conoce con precisión el estado real del proyecto. No hay que esperar durante meses para saber cómo van las cosas y las reuniones de seguimiento pueden ser cortas y precisas.

Todos estos cambios entonces son integrados en un entorno en particular, comúnmente conocido como entorno de integración, donde cada cierto período de tiempo se ejecutan pruebas de regresión o de integración. Este entorno contiene siempre la última versión del código (no la última versión productiva) y puede utilizarse a su vez como entorno de pruebas y QA.



Despliegue continuo (CD)

El despliegue continuo o entrega continua es **la capacidad de llevar cambios** de todo tipo (incluyendo nuevas funcionalidades, cambios de configuración, corrección de errores...) **al entorno de producción** o a manos de los usuarios finales de forma **segura**, lo más **automática** posible, **rápida** y **continua**.

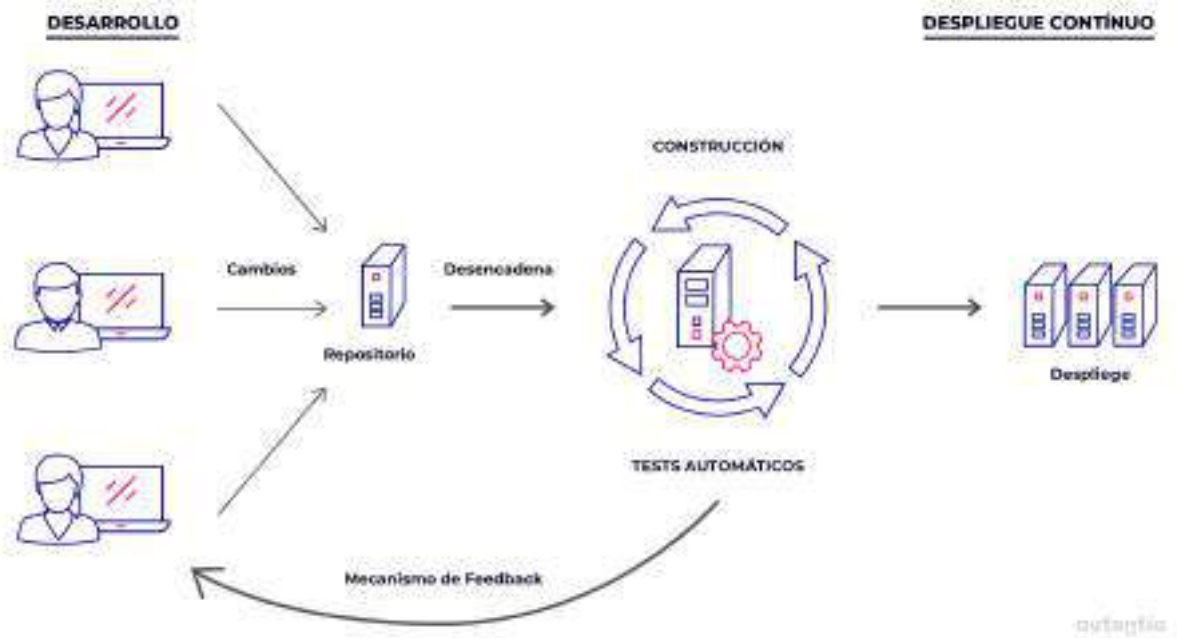
El objetivo es hacer **despliegues predecibles, automáticos y rutinarios** en cualquier momento, ya sea de un sistema distribuido a gran escala, un entorno de producción complejo, un sistema integrado o una aplicación.

Logramos todo esto asegurando que nuestro código esté siempre en un estado desplegable, incluso cuando estamos trabajando con equipos de miles de desarrolladores que realizan cambios a diario. Es decir, cualquier cambio que subamos ni rompe la funcionalidad ya existente ni introduce algún tipo de error que pueda llegar a causarle problemas a un usuario final.

¿Por qué es importante la entrega continua?

La gente asume que si queremos desplegar software con frecuencia, debemos aceptar niveles más bajos de estabilidad y calidad en nuestros sistemas. **No tenemos por qué hacer despliegues renunciando a la calidad y estabilidad** del proyecto si tenemos un sistema bien configurado, con automatización de los procesos de despliegue, minimizando la intervención humana, mediante integración continua por ejemplo. Al automatizar todos esos procesos conseguiremos un sistema de despliegue que **siempre funcionará igual** y que **mantendrá la calidad** a la vez que **mejora los tiempos**, pues las máquinas son más rápidas que el hombre. ¿Te imaginas subir por FTP una aplicación para instalarla en 200 servidores? Esta capacidad de hacer entrega continua proporciona una ventaja competitiva increíble para los clientes, puesto que cualquier cambio que se requiera se podrá desplegar en minutos una vez esté implementado y probado.




DESARROLLO
DESPLIEGUE CONTÍNUO
CONSTRUCCIÓN
Cambios
Repositorio
Desencadena
Despliege
TESTS AUTOMÁTICOS
Mecanismo de Feedback

El planteamiento de GitOps



La idea detrás de **GitOps** fue en su inicio desarrollada y compartida por la compañía WeaveWorks. Se define como un sistema de despliegue basado en infraestructura como código que tiene como pieza central la herramienta Git. Es una **evolución de las mejores prácticas de DevOps e infraestructura como código (IaC)**. Por ponerlo de forma simple, GitOps es un 90% de buenas prácticas y un 10% de nuevos conceptos. Sus principales características son:

- Entorno definido mediante lenguaje declarativo.
- Git como fuente de la verdad.
- Despliegues automáticos bajo aprobación.
- Uso de agentes para asegurar que el despliegue ha sido correcto.

Con GitOps todos los despliegues los tenemos versionados en un repositorio de configuración de Git. El repositorio se modifica a través de pull requests, cuyos cambios tenemos que aprobar para luego unirse a la rama principal y de manera automática se configurará la infraestructura reflejando el estado del repositorio. Este **flujo contínuo de pull requests es**

la esencia de GitOps.

Los beneficios de este planteamiento son muchos. Nos proporciona transparencia y claridad gracias a trabajar con un repositorio central. El control de versiones nos da un histórico de la configuración de la infraestructura y podemos dar marcha atrás a cambios que rompen la funcionalidad existente o introducen un error. Mejora la productividad de nuestro equipo al poder experimentar sin miedo con nuevas configuraciones. Y finalmente la facilidad de adoptar como metodología al usar herramientas que nos son familiares.

En un pipeline CI/CD tradicional se realiza el proceso de construcción y a continuación el proceso de despliegue en el que llevamos los cambios al entorno de producción. En cambio, el pipeline de GitOps **separa el proceso de construcción del proceso de despliegue**. El proceso de construcción es idéntico, pero a la hora del despliegue existe un operador que monitoriza si hay cambios en el repositorio de configuración y cuando existan los va a llevar al entorno de producción. Este operador se encuentra entre el pipeline y nuestro orquestador, que podría ser Kubernetes u otra solución. Una explicación más en profundidad la tenemos en este artículo.



Beneficios de la integración y la entrega continua (CI/CD)

Las prácticas de la integración continua y entrega continua nos ayudan a lograr importantes beneficios:

- **Releases (lanzamientos de incrementos o mejoras del código) de bajo riesgo.** Hacer que los despliegues de software sean indoloros, procesos de bajo riesgo que se pueden realizar en cualquier momento, bajo demanda o automáticamente, según sea necesario.
- **Time to market (tiempo que pasa entre que surge la necesidad de aplicar un cambio hasta que está desplegado en producción para que pueda ser usado por el cliente) más rápido.** No es raro que la fase de integración, prueba y solución de posibles errores en el ciclo de entrega de software consuma semanas o incluso un par de meses. Cuando los equipos trabajan juntos para automatizar los procesos de creación de los artefactos, el sistema de despliegue, la configuración de los entornos y la ejecución de tests, los desarrolladores pueden trabajar de forma más eficiente, ya que tienen una red de seguridad gracias a todo el trabajo realizado y se evitan tener que hacer con cada cambio el mismo tipo de pruebas manuales que consumen tiempo y pueden conllevar errores.
- **Mejor calidad**. Cuando los desarrolladores tienen herramientas automatizadas de tests regresivos (comprueban que un cambio nuevo no quite funcionalidad existente), los equipos tienen la libertad de centrar su esfuerzo en otras actividades como por ejemplo, mejorar la calidad del código haciendo ejercicios de refactorización (simplificar la funcionalidad quitándole complejidad al código mejorando su lectura), hacer pruebas de usabilidad, rendimiento o seguridad. Al construir un pipeline de integración continua, estas actividades se pueden realizar continuamente durante todo el proceso de entrega, asegurando que la calidad se incorpore desde el principio.
- **Costes más bajos**. Cualquier producto o servicio de software exitoso evolucionará significativamente a lo largo de su vida útil. Al invertir en la construcción, prueba, implementación y configuración automática del entorno, reducimos sustancialmente el coste de realizar y entregar cambios incrementales al software al eliminar muchos de

los costes fijos asociados al proceso de lanzamiento.

- **Mejores productos.** La entrega continua hace que sea económico trabajar en pequeños ciclos de semanas (normalmente de 2 o 3 semanas) llamados sprints, donde se define una funcionalidad a cubrir en ese tiempo. Al final de cada ciclo se produce un entregable y el cliente decide si lo quiere llevar al entorno de producción. De esta manera podemos obtener comentarios de los usuarios o el cliente a lo largo del ciclo de vida de entrega. Gracias a este enfoque podemos probar ideas con el cliente antes de desarrollar funciones completas. Esto significa que podemos evitar el problema recurrente de desarrollar una característica (feature) que creamos que ofrece un valor y que, una vez entregada al cliente, este compruebe que realmente la característica no aporta el valor esperado al negocio (otra vez la idea de “Falla rápido, falla barato”).
- **Equipos más felices.** Al ser todo el proceso más rápido, menos dado a errores y más automatizado, hacer cambios es menos doloroso, con lo que estaremos más atentos a las necesidades del negocio, habrá menos miedo al cambio y se reducirá el agotamiento del equipo. Además, cuando lanzamos con más frecuencia, los equipos de entrega de software pueden interactuar más activamente con los usuarios, aprender qué ideas funcionan y cuáles no y ver de primera mano los resultados del trabajo que han realizado. Al eliminar las actividades dolorosas de bajo valor asociadas con la entrega de software, podemos centrarnos en lo que más nos importa: **entregar valor al cliente**.

Un contraejemplo de las consecuencias de no utilizar este tipo de prácticas podemos verlo en el siguiente enlace donde se muestran evidencias de que puede salir muy caro hacer los procesos de forma manual. Esta vez en concreto se perdieron **450 millones** de dólares en 45 minutos, **por subir cambios de forma manual** en tan solo 7 de los 8 servidores. Si hubiera estado automatizada la tarea, este error, a todas luces humano, no hubiera sucedido.

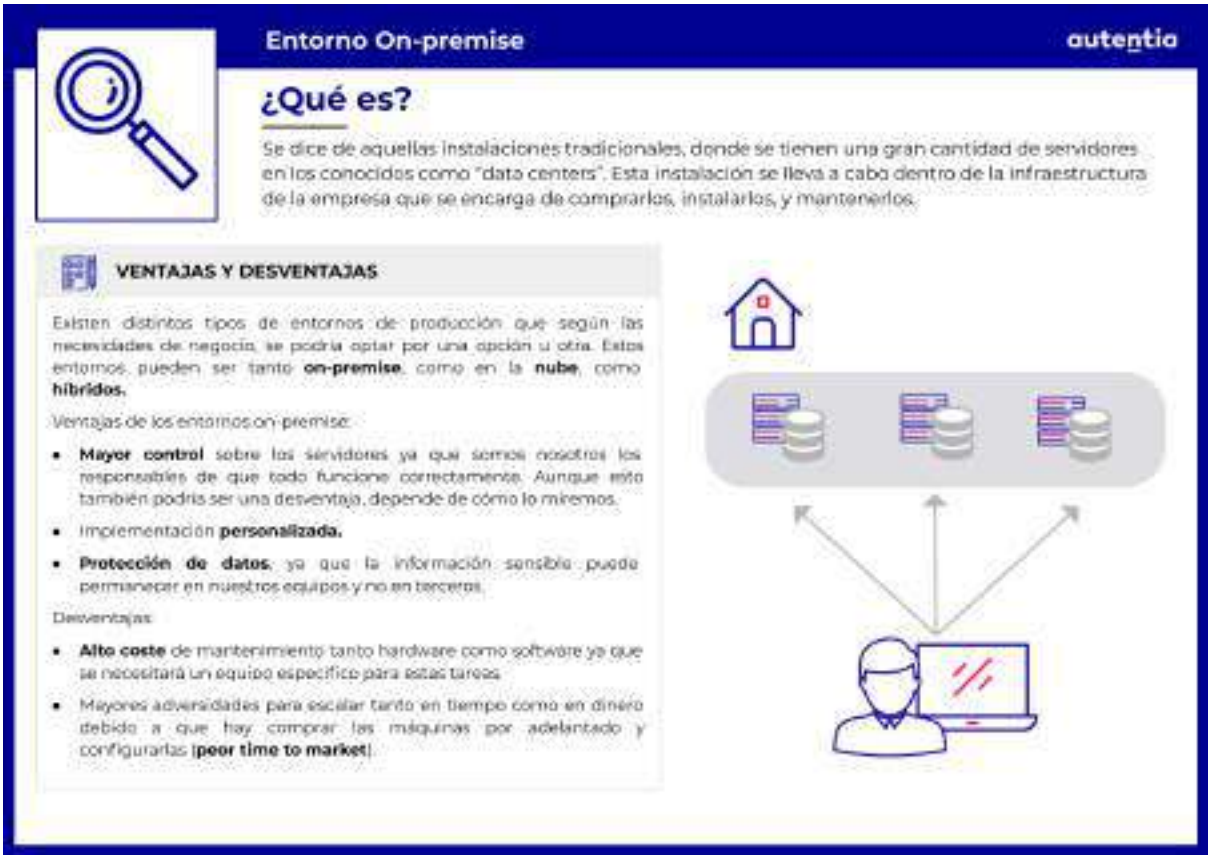
Infraestructura como código (IaC)

Ahora supongamos que nuestra empresa tiene una infraestructura montada, con bases de datos, servidores web y de aplicaciones, aplicaciones en distintos lenguajes, dominios, redes, firewalls, DNSs, etc., y se nos pide migrar toda esa infraestructura, por cambio de equipo físico (hardware) o proveedor de cloud, o incluso se nos solicita crear nuevas instancias de parte de esa infraestructura para poder atender la creciente cantidad de peticiones a un sitio web (escalado horizontal).



Esto nos supondría crear y configurar uno o varios servidores a mano, teniéndonos que apoyar en alguna guía de cómo hacerlo, y eso con suerte de que tal guía exista y no tengamos que crear nosotros dicha guía. Da igual que estos servidores sean virtuales (VPS - virtual private server) o físicos, situados en alguna instalación de la empresa (on premise). Además hay que agregar claves SSH, certificados, crear registros DNS, configurar redes, firewalls, dominios, balanceadores de carga, proxys, servidores web y de

aplicaciones. Cada uno con tecnologías distintas como pueden ser PHP, MySQL, Docker, Java, etc. Bastante trabajo, ¿verdad?



Ahora imaginemos que podemos tener toda esta infraestructura definida en ficheros, con todas las piezas (bases de datos, proxies, firewalls, servidores web y de aplicaciones, etc) que necesitamos para que nuestra aplicación funcione, y que dichos ficheros los tenemos versionados en nuestro repositorio de código, con un historial de cambios realizados, dándonos la posibilidad de automatizar el despliegue de toda o parte de la infraestructura y de volver fácilmente a una versión anterior en caso de un fallo. Esto es lo que se conoce por **Infraestructura como Código (IaC - Infrastructure as Code)**.

Una definición un poco más completa sería que la Infraestructura como Código es la administración de piezas de infraestructura (redes, máquinas virtuales, balanceadores de carga, topologías de red, servidores web y de aplicaciones, bases de datos, etc.) en un modelo descriptivo, el cual se puede versionar en el mismo sistema de versionado utilizado por el equipo de desarrollo. Es un método de aprovisionamiento y gestión de

infraestructura mediante código, que hace que la tarea de crear, modificar y mantener instancias de infraestructura sea más fácil y rápida ya que utilizando ese código y lanzando los mismos comandos conseguiremos el objetivo. Toda esta administración y configuración de infraestructura, o modelos, se termina reflejando en ficheros, comúnmente en formato Yaml o JSON, que son versionados para poder tener una historia con todos los cambios que se fueron realizando.

Siguiendo con el principio de que el mismo código fuente genera el mismo binario, un modelo de IaC genera el mismo entorno cada vez que es aplicado.

IaC nos permite reforzar la consistencia mediante la representación del estado deseado de un entorno mediante código, reduciendo la deriva de la configuración. Esto se refleja en un desarrollo más rápido y eficiente, con una minimización de riesgos, permitiendo la reusabilidad y la automatización. Como resultado, podremos ofrecer entornos estables rápidamente y a escala, mejorando el tiempo de salida a producción / mercado (time to production / market).



Configuración como código (CaC)

La Configuración como Código (CaC - Configuration as Code) puede parecernos, en primera instancia, algo igual o muy similar a la infraestructura como código, pero no lo es. IaC es la administración y aprovisionamiento de infraestructura mediante código, en lugar de manualmente, para crear y configurar sistemas y/o servicios.

La configuración como código es un proceso para administrar datos de configuración de una aplicación. Claramente, hay algunas áreas donde IaC y CaC se superponen, pero ambas administran dos aspectos distintos del entorno.

Una definición más precisa de CaC sería: *"**La configuración como código es la migración formal de la configuración entre entornos, respaldada por un sistema de control de versiones**"*.

Ahora imaginemos que un pequeño cambio en la configuración de nuestra aplicación, implica un redespliegue de la misma. Volver a desplegar una aplicación completa, en todas las instancias o entornos que tengamos dependiendo de nuestra infraestructura, por cambiar solo una propiedad en la configuración no tiene mucho sentido y puede llevar mucho tiempo. Es una práctica bastante frecuente (dejemos el debate de si es buena o mala para las charlas de café) mantener repositorios y pipelines separados para la configuración, para no tener que caer en el redespliegue de la aplicación completa si solo ha habido un cambio de configuración.

Aunque no nos hayamos percatado, ya ponemos en práctica ciertos conceptos de CaC en nuestro trabajo diario. El ejemplo tal vez más claro es en las aplicaciones Java con Maven, donde tenemos ficheros XML para el manejo de dependencias, ficheros de properties (o Yaml también en versiones más recientes) para el manejo de atributos de configuración, etc. Estos ficheros están versionados junto con el código, y se despliegan junto con el mismo. En aplicaciones con poca configuración o que no cambie con mucha frecuencia, este enfoque es más que suficiente.

Pero también podemos hacer esto por ejemplo con una aplicación desplegada en Kubernetes. Toda su configuración la podemos depositar en un repositorio de configuración por entorno independiente de la aplicación.

En este caso, puede suceder que, por ejemplo, debamos cambiar la dirección de acceso a proveedor externo en un determinado entorno. Resulta muy útil poder hacer un cambio únicamente de configuración en el entorno deseado, sin tener que compilar y desplegar toda la aplicación de nuevo.